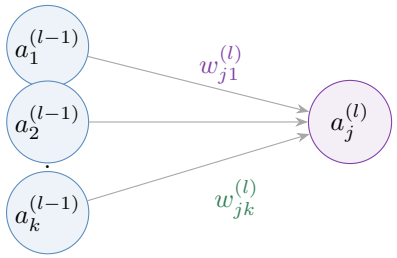
Neural Networks and Backpropagation

1 Neural Network Structure

A neuron computes a weighted sum of its inputs, adds a bias, and applies an activation function σ:

$$z_j^{(l)} = \sum_k w_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)}, \qquad \boxed{a_j^{(l)} = \sigma(z_j^{(l)})}. \tag{1}$$

Here $w_{jk}^{(l)}$ connects neuron k in layer $l-1$ to neuron j in layer l. The input is $\mathbf{a}^{(0)} = \mathbf{x}$.



$$z_j^{(l)} = \mathbf{w}_j^{(l)} \cdot \mathbf{a}^{(l-1)} + b_j^{(l)}$$
$$a_j^{(l)} = \sigma(z_j^{(l)})$$

For an entire layer, the same calculation is written as

$$\boxed{\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})}, \quad W^{(l)} = \begin{bmatrix} w_{11}^{(l)} & \cdots & w_{1n}^{(l)} \\ \vdots & \ddots & \vdots \\ w_{m1}^{(l)} & \cdots & w_{mn}^{(l)} \end{bmatrix}.$$

The activation function acts elementwise. A layer with n inputs and m neurons has $m(n+1)$ trainable parameters (weights and biases).

2 Activation Functions

Activation functions introduce nonlinearity.

$$\textbf{Sigmoid:} \quad \sigma(x) = \frac{1}{1+e^{-x}}, \quad \sigma'(x) = \sigma(x)(1-\sigma(x)), \quad 0 < \sigma(x) < 1.$$

$$\textbf{ReLU:} \quad \mathrm{ReLU}(x) = \max(0, x) = \begin{cases} x, & x > 0, \\ 0, & x \le 0. \end{cases}$$

Sigmoid is monotone with horizontal asymptotes $y = 0$ and $y = 1$. ReLU (and variants) is common in hidden layers; sigmoid remains useful for binary-classification outputs.

3 Loss and Gradient Descent

For output activations $a_j^{(L)}$ and targets y_j, the squared-error loss for one example is

$$\boxed{C = \sum_{j=1}^{m} (a_j^{(L)} - y_j)^2.}$$

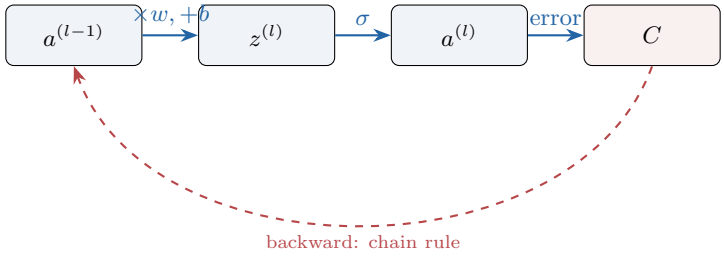
For a dataset with N examples, define the mean loss $J(\theta) = \frac{1}{N}\sum_{i=1}^{N} C_i(\theta)$, where θ contains all weights and biases. Gradient descent updates

$$\boxed{\theta \leftarrow \theta - \eta \nabla J(\theta)},$$

where $\eta > 0$ is the learning rate. The negative gradient points in the direction of steepest local decrease; it does not guarantee a global minimum.

4 Backpropagation: One Output Neuron

The forward computation is a chain of functions:



For $z^{(l)} = w^{(l)}a^{(l-1)} + b^{(l)}$, $a^{(l)} = \sigma(z^{(l)})$, and $C = (a^{(l)} - y)^2$,

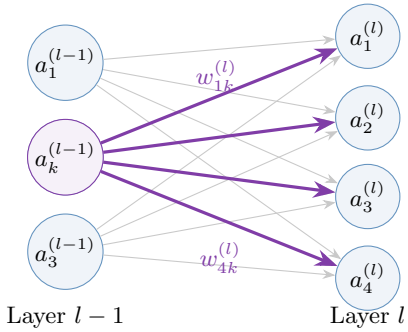
$$\frac{\partial C}{\partial w^{(l)}} = \underbrace{\frac{\partial C}{\partial a^{(l)}}}_{2(a^{(l)}-y)} \underbrace{\frac{\partial a^{(l)}}{\partial z^{(l)}}}_{\sigma'(z^{(l)})} \underbrace{\frac{\partial z^{(l)}}{\partial w^{(l)}}}_{a^{(l-1)}} = \boxed{2(a^{(l)} - y)\sigma'(z^{(l)})a^{(l-1)}},$$

$$\frac{\partial C}{\partial b^{(l)}} = 2(a^{(l)} - y)\sigma'(z^{(l)}).$$

For the mean loss, average the individual-example gradients: $\frac{\partial J}{\partial w} = \frac{1}{N}\sum_{i=1}^{N} \frac{\partial C_i}{\partial w}$.

5 Backpropagation: Multiple Layers

A hidden neuron can influence the loss through *multiple* neurons in the next layer. Their gradient contributions must be summed.



Define the local error signal $\delta_j^{(l)} := \frac{\partial C}{\partial z_j^{(l)}}$. For the output layer (squared-error loss),

$$\boxed{\delta_j^{(L)} = 2(a_j^{(L)} - y_j)\sigma'(z_j^{(L)})}.$$

For a hidden neuron k in layer $l-1$,

$$\frac{\partial C}{\partial a_k^{(l-1)}} = \sum_j \underbrace{\frac{\partial C}{\partial z_j^{(l)}}}_{\delta_j^{(l)}} \underbrace{\frac{\partial z_j^{(l)}}{\partial a_k^{(l-1)}}}_{w_{jk}^{(l)}} = \sum_j w_{jk}^{(l)} \delta_j^{(l)}.$$

Applying the activation derivative gives the central backpropagation rule:

$$\boxed{\delta_k^{(l-1)} = \sigma'(z_k^{(l-1)}) \sum_j w_{jk}^{(l)} \delta_j^{(l)}}.$$

Once $\delta_j^{(l)}$ is known, the parameter gradients follow:

$$\boxed{\frac{\partial C}{\partial w_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)}, \qquad \frac{\partial C}{\partial b_j^{(l)}} = \delta_j^{(l)}}.$$

6 Training Loop

1. Initialize weights and biases.
2. **Forward:** compute activations and predictions.
3. **Loss:** measure the prediction error.
4. **Backward:** use the chain rule to compute parameter gradients.
5. **Update:** apply gradient descent; repeat.

$$\boxed{\text{Forward} \longrightarrow \text{Loss} \longrightarrow \text{Backward} \longrightarrow \text{Update}}$$